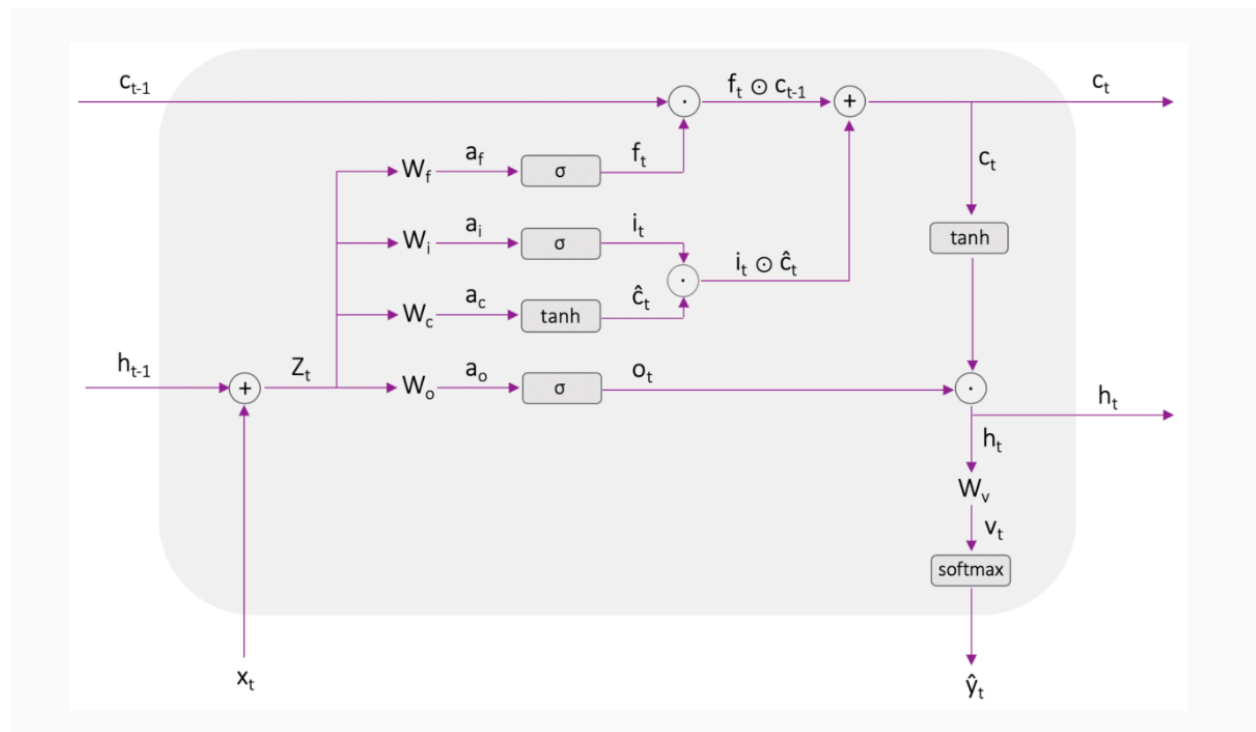


Catatan Master: Arsitektur & Pembelajaran LSTM

1. Filosofi Dasar

LSTM (**Long Short-Term Memory**) diciptakan untuk menyelesaikan masalah **Vanishing Gradient** (gradien hilang) pada RNN biasa. Ia memiliki "Jalan Tol" informasi yang disebut **Cell State** agar informasi penting bisa diingat dalam jangka waktu yang sangat lama.



2. Anatomi Internal (The 4 Gates)

Setiap "kotak" LSTM memiliki 4 layer neural network (barisan neuron) yang bekerja secara paralel pada setiap langkah waktu (t).

Gate	Simbol	Fungsi Utama	Aktivasi
Forget Gate	f_t	Memutuskan info lama mana yang harus dibuang dari memori.	Sigmoid (σ)

Input Gate	i_t	Memutuskan info baru mana yang layak masuk ke memori.	Sigmoid (σ)
Candidate	$\tilde{C}_t$	Membuat materi/ide memori baru untuk ditambahkan.	Tanh
Output Gate	o_t	Memutuskan bagian memori mana yang akan dikeluarkan ke luar.	Sigmoid (σ)

3. Operasi Matematika Maju (Forward Pass)

Input gabungan Z_t terdiri dari $[h_{t-1}, x_t]$.

1. **Hitung Gates:** Semuanya menggunakan perkalian matriks bobot ($W \cdot Z_t + b$).
2. **Update Memori (Cell State):**

$$C_t = (f_t \odot C_{t-1}) + (i_t \odot \tilde{C}_t)$$

- *Note:* $\odot$ adalah **Hadamard Product** (perkalian elemen-per-elemen).
3. **Hitung Output (Hidden State):**

$$h_t = o_t \odot \tanh(C_t)$$

4. Mekanisme Belajar (Backpropagation Through Time)

Saat AI salah menebak, pesan error (gradien) harus mengalir mundur.

A. Aliran di Cell State (The Highway)

Gradien meluncur di jalur atas tanpa hambatan matriks berat. Ia hanya dikalikan dengan **Forget Gate** (f_t).

- Jika $f_t = 1$, gradien lewat 100% (Ingat terus).

- Jika $f_t = 0$, gradien terputus (Lupakan).

B. Percabangan Gradien (Branching)

Dari Hidden State (h_t), gradien pecah menuju ke **Output Gate** dan **Cell State**. Urutan mundurnya dari h_t ke C_t :

1. **Kali** o_t : Melewati kran output.
2. **Kali** $\tanh'(C_t)$: Menghitung sensitivitas/pengaruh C_t terhadap output.

5. Mengapa Harus Dikali Turunan (σ' , $\tanh'$)?

Turunan fungsi aktivasi bertindak sebagai **Filter Sensitivitas**.

- Ia memberi tahu gradien: "Apakah input ini berada di area yang sensitif untuk diubah (slope besar) atau sudah mentok di ujung (slope hampir 0)?"
- Tanpa ini, gradien tidak tahu kekuatan update yang pas untuk setiap bobot.

6. Update Bobot (The Final Action)

Setelah gradien sampai di depan Matriks Bobot (W), kita menghitung "Saran Perubahan" (∇W):

$$\nabla W = \delta_{\text{gate}} \cdot Z_t^T$$

- **Kenapa Transpose (Z_t^T)?** Agar dimensi gradien (vektor berdiri) dan input (vektor tidur) menghasilkan matriks yang ukurannya sama dengan W .
- **Update Rule (Manual):**

$$W_{\text{baru}} = W_{\text{lama}} - (\text{Learning Rate} \times \nabla W)$$

Pesan Kunci: LSTM belajar dengan cara menyebarkan "penyesalan" (error) ke setiap gate. Setiap gate kemudian menyesuaikan tombol-tombolnya (bobot)

berdasarkan seberapa sensitif fungsi mereka saat itu dan seberapa besar input yang masuk.

Selamat! Kamu sudah menyelesaikan maraton pembelajaran LSTM hari ini.

Apakah rangkuman ini sudah cukup lengkap untuk catatanmu? Jika sudah, sesuai janji di awal, **apakah kamu ingin mencoba kuis singkat 3 pertanyaan** untuk mengetes apakah catatan ini sudah benar-benar meresap di kepala? (Tenang, kuisnya seru kok!)

Atau ada satu bagian terakhir yang ingin kamu tanyakan sebelum kita akhiri sesi hebat ini?